

Topcoder Skills Recommender

A powerful CLI tool that analyzes GitHub user profiles and recommends matching skills from Topcoder's standardized skill database. Uses multiple sophisticated matching algorithms to provide evidence-based skill recommendations.

Features

- **Deep GitHub Analysis:** Analyzes repositories, commits, pull requests, languages, and topics
- **Multiple Matching Algorithms:** Choose from 6+ specialized matchers or use hybrid mode for best accuracy
- **Evidence-Based Recommendations:** Every skill match includes verifiable evidence from your GitHub activity
- **Confidence Scoring:** Intelligent scoring system (0-100%) based on contribution patterns and frequency
- **Flexible CLI:** Extensive configuration options for customized analysis workflows
- **Fast & Efficient:** Local skill matching with O(1) index lookups (~2-5 seconds per user)
- **Plug-and-Play Architecture:** Easy to add custom matching algorithms via simple interface
- **Rich Documentation:** Comprehensive architecture diagrams, flow charts, and guides

Quick Example

```
npm start -- --matcher hybrid --min-confidence 60
```

Output:

 Topcoder Skill Recommendations for octocat

1. JavaScript (98% confidence) ★★★★★

Evidence:

- Language: frontend-app (42 contributions)
- Language: backend-api (38 contributions)
- Repository: js-toolkit (topic: javascript)

Found by: Language Matcher, Repository Matcher

2. React (95% confidence) ★★★★★

Evidence:

- Language: react-dashboard (56 contributions)
- Repository: react-components (description: "React UI library")

Found by: Language Matcher, Hybrid Matcher

Quick Start

Installation

```
git clone https://github.com/kondekarshubham123/Topcoder-Github-Skill-Import.git
cd Topcoder-Github-Skill-Import
npm install
npm run build
```

Configuration

Create a `.env` file with your GitHub OAuth credentials:

```
GITHUB_CLIENT_ID=your_client_id
GITHUB_CLIENT_SECRET=your_client_secret
```

Getting OAuth Credentials: 1. Visit <https://github.com/settings/developers> 2. Click "New OAuth App" 3. Set Homepage URL: <http://localhost> 4. Set Authorization callback URL: <http://localhost> 5. Copy Client ID and Client Secret to `.env`

First Run

```
npm start
```

The CLI will: 1. Authenticate via GitHub OAuth (first time only) 2. Fetch your GitHub profile and repositories 3. Download Topcoder's skill database (~10,000 skills) 4. Run the hybrid matcher (recommended) 5. Display skill recommendations with evidence

Usage

Basic Command

```
npm start -- [options]
```

The CLI uses the authenticated GitHub user automatically. No username parameter needed.

Matcher Algorithms

Matcher	Description	Best For	Speed
hybrid (default)	Combines all matchers with weighted scoring	Most accurate, recommended	Medium
language	Matches based on programming languages	Developers with clear language focus	Fast
repository	Analyzes repo names, descriptions, topics	Project-based analysis	Fast
commit	Deep analysis of commit messages and files	Detailed code contribution patterns	Slow
pr	Analyzes pull request metadata	Open source contributors	Medium
all	Runs all matchers sequentially	Maximum coverage	Slowest

Examples

Analyze with hybrid matcher:

```
npm start -- --matcher hybrid
```

Language-only matching (fastest):

```
npm start -- --matcher language
```

High-confidence skills only:

```
npm start -- --min-confidence 75
```

Limit number of results:

```
npm start -- --max-skills 10
```

JSON output for integration:

```
npm start -- --output json --output-file results.json
```

Analyze top repos only (faster):

```
npm start -- --max-repos 20
```

Skip deep analysis (much faster):

```
npm start -- --no-deep-analysis
```

Verbose logging:

```
npm start -- --verbose
```

All CLI Options

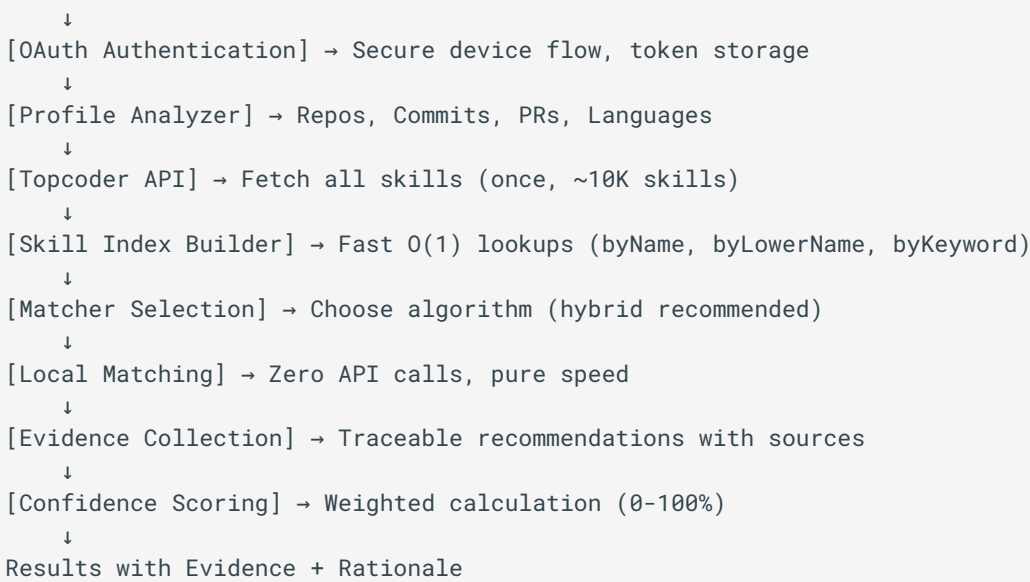
-m, --matcher <type>	Matcher algorithm (default: hybrid) Options: language, repository, commit, pr, hybrid, all
-r, --max-repos <number>	Maximum repositories to analyze (default: 50)
-c, --max-commits <number>	Maximum commits per repo (default: 30)
-s, --max-skills <number>	Maximum skills to display (default: 15)
--no-deep-analysis	Skip commit/PR analysis (faster)
--min-confidence <number>	Filter results by confidence 0-100 (default: 30)
-v, --verbose	Show detailed logging
-o, --output <format>	Output format: text or json (default: text)
--output-file <path>	Save results to file
-h, --help	Show help

 Architecture

System Overview

The system follows a streamlined pipeline from GitHub authentication to skill recommendations:

```
GitHub User
```



See [ARCHITECTURE.md](#) for detailed Mermaid diagrams including: - Complete data flow sequence diagrams - Component architecture graphs - Skill index structure visualization - Matcher interface class diagrams - Performance optimization strategies

Matching Algorithms

- 1. Language Matcher** (Most Reliable, Fastest) - Matches programming languages directly to skills - **Exact match:** 95% base confidence (e.g., "JavaScript" === skill.name) - **Partial match:** 75% confidence (e.g., "JavaScript" in skill.name) - **Keyword match:** 60% confidence (skill keywords contain "javascript") - **Frequency boost:** +2% per repository (max +20%) - **Evidence:** Top 5 repositories using each language - See [DESIGN_FLOW.md](#) Section 4 for algorithm flowchart
- 2. Repository Matcher** (Fast, Metadata-focused) - Analyzes repo names, descriptions, and topics - Tokenizes metadata into keywords - Searches `skillIndex.byKeyword` for matches - **Confidence formula:** $\min(30 + (\text{repos} \times 5) + (\text{topics} \times 8) + (\text{descriptions} \times 3), 85)$ - **Evidence:** Repositories with matching metadata - Good for project-type skills ("Web Development", "DevOps")
- 3. Commit Matcher** (Deep Analysis, Slower) - Deep analysis of commit messages and file changes - File extension mapping (`.js` → JavaScript, `.py` → Python, etc.) - Extracts keywords from commit messages - **Confidence formula:** $\min(40 + (\text{commits} \times 3) + (\text{file_changes} \times 2), 80)$ - **Evidence:** Specific commits and file paths - Best for detailed contribution patterns
- 4. Pull Request Matcher** (Collaboration-focused) - Analyzes PR titles, descriptions, and labels - Tokenizes PR metadata - **Confidence formula:** $\min(20 + (\text{PRs} \times 8) + (\text{labels} \times 5), 75)$ - **Evidence:** Pull requests with matching content - Ideal for open source contributors
- 5. Hybrid Matcher** (Recommended, Most Accurate) - Runs all matchers in parallel using `Promise.all()` - Combines results with weighted scoring: - **Language:** 1.0 (most reliable) - **Repository:** 0.7 - **Commit:** 0.5 - **Pull Request:** 0.4 - **Weighted confidence:** $\frac{\sum(\text{confidence} \times \text{weight})}{\sum(\text{weights_used})}$ - Merges evidence from all sources - Deduplicates and creates comprehensive rationale - See [DESIGN_FLOW.md](#) Section 5 for weighted calculation diagram
- 6. All Matchers** (Maximum Coverage) - Runs matchers sequentially (not parallel) - Combines all results without weighting - Maximum coverage but longest runtime - Useful for comprehensive exploration

Skill Index Performance

The system uses an in-memory index strategy for optimal performance:

Before (API-based approach): - 100+ API calls per user (fuzzy/semantic search for each language) - 30-60 seconds per analysis - Network-dependent reliability - Rate limit vulnerable

After (Local index approach): - 1 API call per session (fetch all skills once) - 2-5 seconds per analysis (5-10x faster) - 100% reliability (no network failures) - O(1) lookup performance via Map structures - Parallel matcher execution enabled

Index Structure: - `byName` : Exact name matches (e.g., "React") - `byLowerName` : Case-insensitive matches (e.g., "react", "javascript") - `byKeyword` : Token-based search (e.g., "front", "web", "database") - `all` : Complete skill array for iteration

See [ARCHITECTURE.md](#) Section 2 for detailed index diagrams.

Adding Custom Matchers

The system is **plug-and-play**. See [PLUGIN_GUIDE.md](#) for step-by-step instructions.

Quick example:

```
import { ISkillsMatcher, SkillMatch, GithubProfile, Evidence } from './ISkillsMatcher';
import { SkillIndex } from '../topcoder';

export class YourMatcher implements ISkillsMatcher {
  name = 'Your Matcher';
  description = 'Your matching strategy';

  async match(profile: GithubProfile, skillIndex: SkillIndex): Promise<SkillMatch[]> {
    // 1. Analyze GitHub data
    // 2. Search skillIndex.byName / byLowerName / byKeyword
    // 3. Calculate confidence
    // 4. Collect evidence
    // 5. Return matches
    return matches;
  }
}
```

Add to CLI in `src/cli.ts`:

```
case 'yourmatcher':
  matcher = new YourMatcher();
  break;
```

Done! No other changes needed.

Documentation

Comprehensive documentation with visual diagrams:

- [ARCHITECTURE.md](#) - Detailed system architecture
- High-level architecture diagrams
- Data flow sequence diagrams
- Component architecture graphs
- Skill index structure (byName, byLowerName, byKeyword maps)
- Matcher interface specifications
- Performance optimization strategies
- Error handling flows
- [DESIGN_FLOW.md](#) - Visual flows and decision trees
- Complete data flow from CLI to output

- Matcher selection decision tree
- Algorithm flowcharts (Language, Hybrid, etc.)
- Confidence score calculation formulas
- Evidence collection process
- Rate limiting strategy
- Memory & performance analysis
- [PLUGIN_GUIDE.md](#) - Step-by-step guide to add custom matchers
- [QUICK_REFERENCE.md](#) - Command cheatsheet and quick reference
- [CLI_USAGE.md](#) - Comprehensive CLI documentation
- [IMPLEMENTATION.md](#) - Implementation details and code structure

Note: All Mermaid diagrams render correctly on GitHub and support rich visualizations.

API Reference

Topcoder Standardized Skills API

Base URL: `https://api.topcoder-dev.com/v5/standardized-skills`

Endpoint Used: - `GET /skills?page=1&perPage=10000` - Fetch all skills (single call)

Documentation: `https://api.topcoder-dev.com/v5/standardized-skills/docs/#/Skills`

GitHub API

Uses GitHub REST API v3 with OAuth authentication: - `/user` - User profile - `/user/repos` - Repositories (paginated) - `/repos/{owner}/{repo}/languages` - Language statistics - `/repos/{owner}/{repo}/commits` - Commit history - `/repos/{owner}/{repo}/pulls` - Pull requests

Rate Limits: - Authenticated: 5,000 requests/hour - Unauthenticated: 60 requests/hour

Use Cases

1. **Personal Skill Assessment:** Discover your strengths based on GitHub activity
2. **Resume Building:** Find verified skills for your resume/portfolio
3. **Topcoder Onboarding:** Quickly identify relevant skills for Topcoder challenges
4. **Team Analysis:** Analyze team members' skills (run for each member)
5. **Hiring & Recruiting:** Assess candidate skills from their public GitHub
6. **Open Source Contribution:** Find skills demonstrated in OSS projects

Confidence Levels

Range	Level	Description
90-100%	★★★★ Expert	Direct language matches, 10+ repos

Range	Level	Description
75-89%	★★★ Advanced	Strong keyword matches, 5-9 repos
60-74%	★ Intermediate	Partial matches, 3-4 repos
40-59%	Beginner	Related keywords, 1-2 repos
0-39%	Exposure	Weak associations, mentioned in commits

Evidence Types

Every skill recommendation includes verifiable evidence:

- **Language:** Direct language usage in repositories
- **Repository:** Repo names, descriptions, topics
- **Commit:** Commit messages and file changes
- **Pull Request:** PR titles, descriptions, labels
- **Topic:** Repository topics/tags
- **File:** File extensions and patterns

Troubleshooting

Authentication failed: - Check `.env` file has correct OAuth credentials - Visit <https://github.com/settings/developers> to verify app settings - Delete `.github-token` and re-authenticate

Rate limit errors: - GitHub limits to 5,000 requests/hour (authenticated) - Use `--max-repos` to reduce API calls - Wait for rate limit to reset (check headers)

No skills found: - Lower `--min-confidence` threshold (try 20-30) - Use `--matcher all` for maximum coverage - Check if user has public repositories

Slow performance: - Use `--no-deep-analysis` to skip commits/PRs - Reduce `--max-repos` (default 50) - Reduce `--max-commits` (default 30) - Use `--matcher language` for fastest results

Contributing

Contributions welcome! Here's how:

1. **Add a new matcher:** See [PLUGIN_GUIDE.md](#) for step-by-step guide
2. **Improve algorithms:** Submit PRs with enhanced confidence scoring formulas
3. **Documentation:** Help improve docs, add examples, fix diagrams
4. **Bug fixes:** Report issues or submit fixes via GitHub Issues
5. **Performance:** Optimize matching speed or memory usage
6. **Testing:** Add test cases and validation scenarios

Repository: <https://github.com/kondekarshubham123/Topcoder-Github-Skill-Import>

Getting Started:

```
git clone https://github.com/kondekarshubham123/Topcoder-Github-Skill-Import.git
cd Topcoder-Github-Skill-Import
npm install
npm run build
```



License

MIT



Links

- **GitHub Repository:** <https://github.com/kondekarshubham123/Topcoder-Github-Skill-Import>
- **Topcoder Skills API:** <https://api.topcoder-dev.com/v5/standardized-skills/docs/>
- **GitHub REST API:** <https://docs.github.com/en/rest>



Acknowledgments

- **Topcoder** for the Standardized Skills API and skill database
- **GitHub** for the comprehensive REST API and OAuth support
- **Open Source Community** for inspiration, feedback, and support
- **Mermaid** for beautiful diagram rendering in documentation